

lab_2-2

December 7, 2025

Write Python code to implement a Logistic Regression Model for Spam detection

Logistic regression is often mentioned in connection to classification tasks. The model is simple and one of the easy starters to learn about generating probabilities, classifying samples, and understanding gradient descent

Logistic regression as mentioned above is mostly used for classification tasks that's why the output of this is in most cases is 1/0, yes/no etc or in form of probability of 1.

In linear regression the main equation is $y = mx + c$.

and here the main equation is $\Rightarrow \sigma(z) = \frac{1}{1+e^{-z}} = h$

where $z = wx + b$

From the above the Likelihood function is $l = \prod_{i=1}^m [h^y][1-h]^{1-y}$

Loss Likelihood Function $\Rightarrow L = \ln(l) = \sum_{i=1}^m y \ln(h) + (1-y) \ln(1-h)$

negative average loss likelihood function $\Rightarrow J = -\frac{1}{m} L$

Now for gradient descent we need 2 functions 1. $\frac{\delta J}{\delta w} = \frac{\delta J}{\delta h} \times \frac{\delta h}{\delta z} \times \frac{\delta z}{\delta w}$

$$= \sum_{i=1}^m \left[\left[\frac{y}{h} - \frac{1-y}{1-h} \right] \times [h(1-h)] \times x \right]$$

$$= \sum_{i=1}^m (y - h)x$$

$$2. \frac{\delta J}{\delta b} = \frac{\delta J}{\delta h} \times \frac{\delta h}{\delta z} \times \frac{\delta z}{\delta b}$$

$$= \sum_{i=1}^m \left[\left[\frac{y}{h} - \frac{1-y}{1-h} \right] \times [h(1-h)] \right]$$

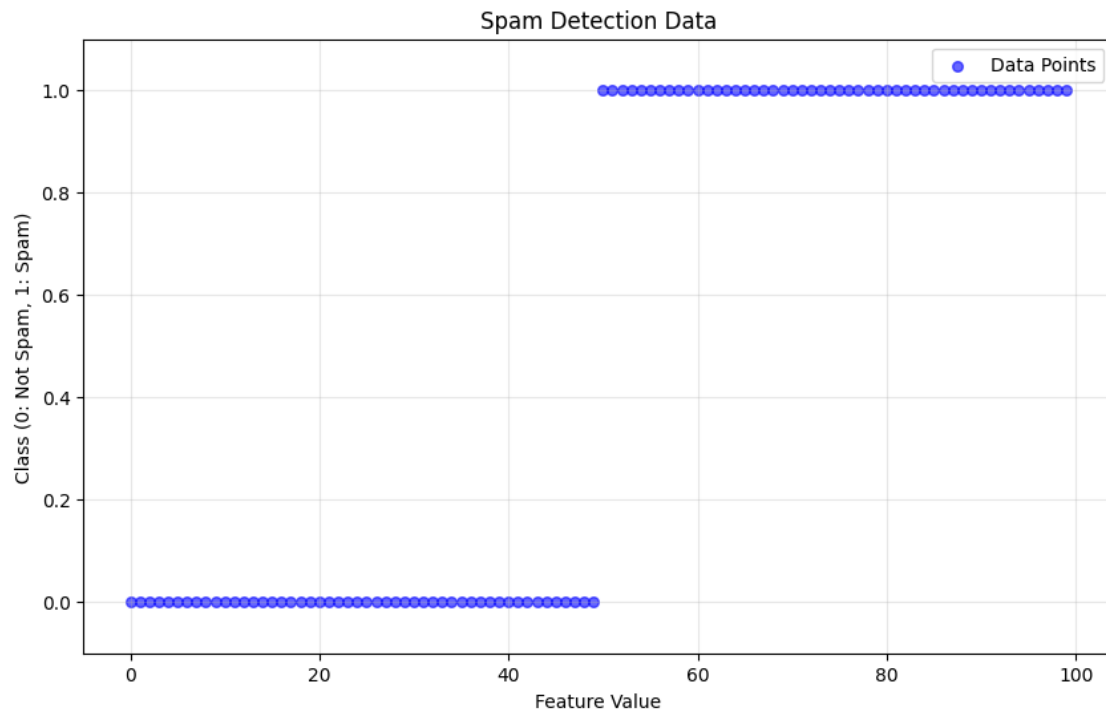
$$= \sum_{i=1}^m (y - h)$$

Logistic Regression with one feature

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
[ ]: x_data = np.array(list(range(100)))
y_data = np.concatenate((np.zeros(50), np.ones(50)), axis = 0)
```

```
[ ]: plt.figure(figsize=(10, 6))
plt.scatter(x_data, y_data, color="blue", label='Data Points', alpha=0.6, s=30)
plt.xlabel('Feature Value')
plt.ylabel('Class (0: Not Spam, 1: Spam)')
plt.title('Spam Detection Data')
plt.legend()
plt.grid(True, alpha=0.3)
plt.ylim(-0.1, 1.1) # Add some padding
plt.show()
```



```
[ ]: def sigmoid(w,b,x):
    return 1 / (1 + np.exp(-((x*w)+b)))
```

```
[ ]: def gradient_decent(w_now, b_now, x, y, L):
    w_grad = 0
    b_grad = 0
    n = len(x)

    for i in range(n):
        h_i = sigmoid(w_now, b_now, x[i])

        w_grad += (1/n) * (y[i] - h_i) * x[i]
        b_grad += (1/n) * (y[i] - h_i)
```

```

w_l = w_now + L * w_grad
b_l = b_now + L * b_grad

return (w_l, b_l)

```

```

[ ]: m = 0
    b = 0
    L = 0.1 # Increased learning rate
    epochs = 5000 # Increased epochs

    for i in range(epochs):
        m, b = gradient_decent(m, b, x_data, y_data, L) # Update m and b!

    print(f"Final w: {m}, Final b: {b}")

```

Final w: 1.142541404978092, Final b: -45.12369357247058

```

[ ]: h = []
    for i in x_data:
        h.append(sigmoid(m, b, i)) # Use m and b, not w and b

```

```

[ ]: h

```

```

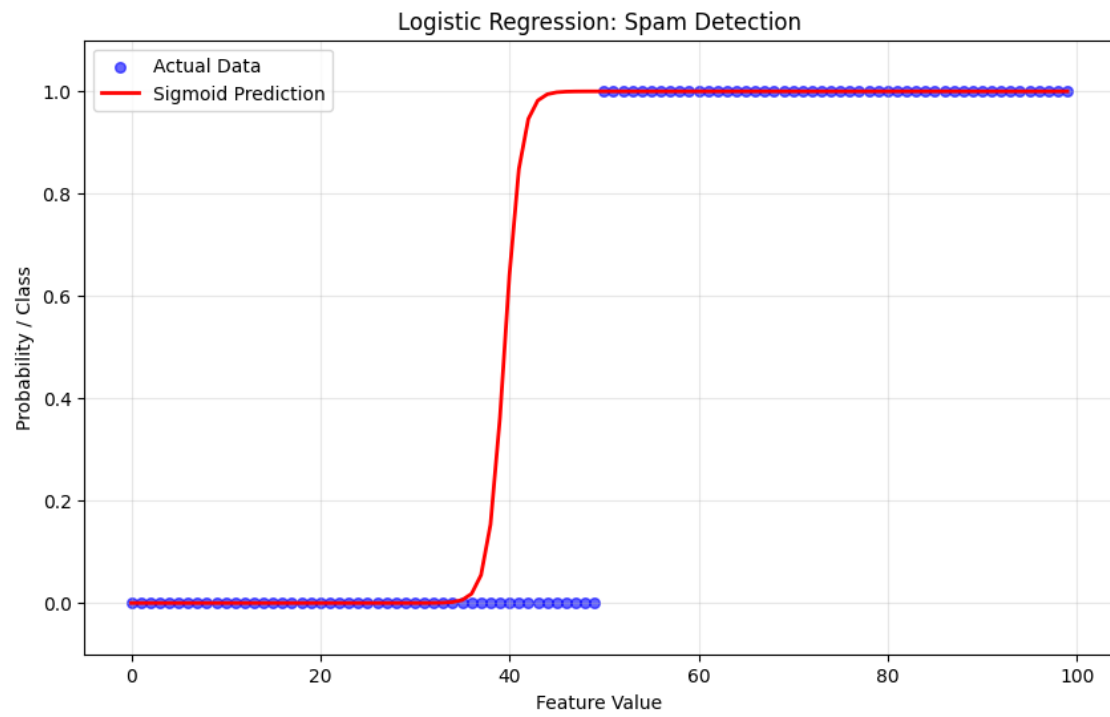
[ ]: [np.float64(2.5294661875406792e-20),
      np.float64(7.929180530305491e-20),
      np.float64(2.4855799295464985e-19),
      np.float64(7.791609186537599e-19),
      np.float64(2.4424550984692604e-18),
      np.float64(7.656424706652217e-18),
      np.float64(2.4000784835460778e-17),
      np.float64(7.523585678542553e-17),
      np.float64(2.35843710322085e-16),
      np.float64(7.393051408602035e-16),
      np.float64(2.317518201167574e-15),
      np.float64(7.264781909258439e-15),
      np.float64(2.277309242382616e-14),
      np.float64(7.13873788672406e-14),
      np.float64(2.237797909344374e-13),
      np.float64(7.01488072895535e-13),
      np.float64(2.198972098237243e-12),
      np.float64(6.893172493797394e-12),
      np.float64(2.1608199152166912e-11),
      np.float64(6.773575897094985e-11),
      np.float64(2.1233296725047522e-10),
      np.float64(6.656054298706651e-10),
      np.float64(2.0864898822895323e-09),
      np.float64(6.540571666516682e-09),
      np.float64(2.0502892288720438e-08),

```

```
np.float64(6.427092326279227e-08),
np.float64(2.0147163302306345e-07),
np.float64(6.315578641766765e-07),
np.float64(1.979757465675999e-06),
np.float64(6.205968708460045e-06),
np.float64(1.9453746641573e-05),
np.float64(6.097961075275232e-05),
np.float64(0.0001911294213179624),
np.float64(0.0005988937938099161),
np.float64(0.0018749701619104086),
np.float64(0.0058540843322915535),
np.float64(0.018124445025150357),
np.float64(0.05469881278416192),
np.float64(0.15353761441550112),
np.float64(0.36248868212412755),
np.float64(0.6405984721416959),
np.float64(0.8481937474083591),
np.float64(0.9459891136864382),
np.float64(0.9821122035025139),
np.float64(0.9942232942717973),
np.float64(0.9981499108543554),
np.float64(0.9994090636041526),
np.float64(0.9998114111045193),
np.float64(0.9999398310450542),
np.float64(0.999980804879786),
np.float64(0.9999938765371722),
np.float64(0.9999980465627306),
np.float64(0.999999376838492),
np.float64(0.9999998012068678),
np.float64(0.9999999365835381),
np.float64(0.9999999797696872),
np.float64(0.999999993546383),
np.float64(0.9999999979412493),
np.float64(0.9999999993432436),
np.float64(0.99999999979049),
np.float64(0.9999999999331648),
np.float64(0.999999999978679),
np.float64(0.9999999999931986),
np.float64(0.9999999999978302),
np.float64(0.9999999999993079),
np.float64(0.9999999999997793),
np.float64(0.9999999999999296),
np.float64(0.9999999999999776),
np.float64(0.9999999999999929),
np.float64(0.9999999999999978),
np.float64(0.9999999999999993),
np.float64(0.9999999999999998),
```

[illegible]

```
[ ]: # Second plot - Data with sigmoid curve
plt.figure(figsize=(10, 6))
plt.scatter(x_data, y_data, color='blue', label='Actual Data', alpha=0.6, s=30)
plt.plot(x_data, h, label='Sigmoid Prediction', color="red", linewidth=2)
plt.xlabel('Feature Value')
plt.ylabel('Probability / Class')
plt.title('Logistic Regression: Spam Detection')
plt.legend()
plt.grid(True, alpha=0.3)
plt.ylim(-0.1, 1.1)
plt.show()
```



[]: